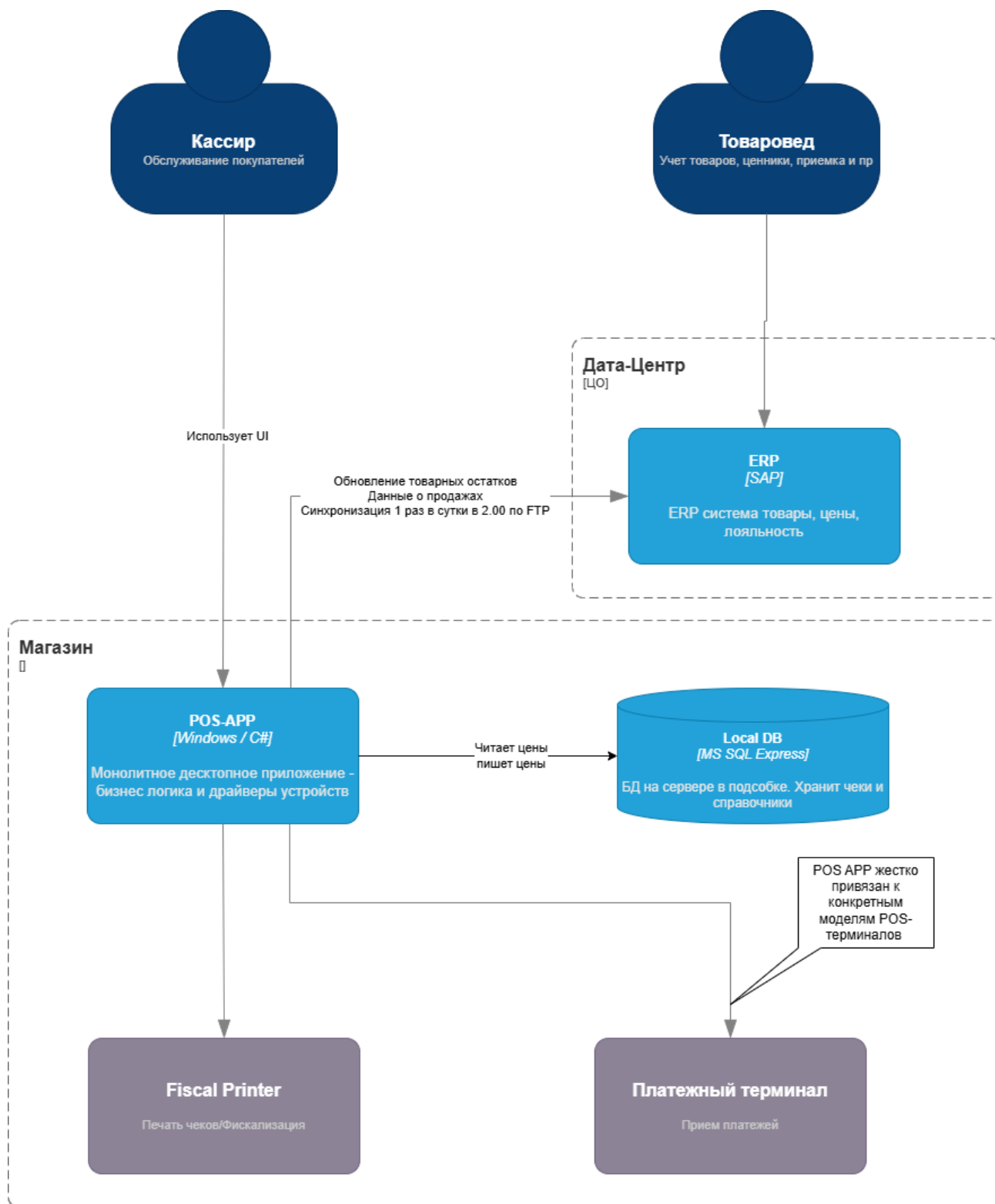


Задание 3. Разработка целевой облачной архитектуры

Проверка схемы AS-IS (Артефакт 1) и её корректировка



Схема, на внутреннем Wiki, содержит критическую неточность – указана интеграция кассы с SAP через REST API в реальном времени, однако из Артефакта 2 следует, что фактически используется ночной обмен файлами по FTP. Это означает, что справочники (цены, остатки) обновляются с задержкой до суток, а локальная БД (MS SQL Express) является «источником истины» для кассового приложения.

Целевая архитектура TO-BE (Cloud-Native + Edge Gateway)

Архитектура строится на принципе «тонкий клиент» + умный бэкенд (облако) .

Добавляются следующие ключевые компоненты:

- Кассовые терминалы в виде тонких клиентов или планшетов;
- Edge Gateway;
- Локальная кеширующая БД SQLite;
- API шлюз для доступа по REST к SAP ERP;

Проблема разнородного оборудования – Hardware Abstraction Layer (Edge Gateway)

В новой архитектуре вся логика работы с железом вынесена в Edge Gateway – небольшой бокс (промышленный мини-ПК на Linux или аналогичное устройство типа Raspberry PI), который находится в магазине, к которому физически подключены платёжные терминалы и др устройства (фискальные регистраторы) по USB или COM портам.

Функции Edge Gateway:

- Кассир на планшете нажимает «Оплата картой». Web POS отправляет на локальный Edge Gateway универсальный REST (HTTP)-запрос на оплату чека.
- Edge Gateway транслирует этот запрос в конкретный протокол терминала (например, ARCUS-2 для Сбера, IDTech для Ingenico) с помощью загружаемых плагинов (драйверов). Аналогично для ККТ – команды печати преобразуются в СОМ-последовательности.
- Ответ от терминала (успех/отказ) конвертируется обратно в универсальный JSON и возвращается на планшет.

Преимущества:

- Кассовое приложение не зависит от модели оборудования – добавление нового терминала требует лишь обновления плагина на Edge Gateway, без адаптации, добавления новых драйверов к кассовому ПР.
- Время адаптации к новому оборудованию сокращается с недель до нескольких дней.

Офлайн-режим и синхронизация касс с облаком

Продажи

Целевая архитектура должна быть разработана с учетом следующих требований:

- Работа магазина при отсутствии связи с головным офисом или облачным сервисом.
- Снижение стоимости поддержки ИТ-инфраструктуры магазинов (ТСО), отказ от «серверов в подсобках»;
- Простота реализации, понятная схема работы систем

С учетом указанных требований, ИТ- архитектура розничного магазина строится исходя из следующих ключевых принципов:

- Бессерверная архитектура (server less) – реализация ключевых функций в кассовом оборудовании магазина. Предлагаемый подход позволит реализовать требуемые функции по хранению/кешированию данных и справочников, необходимых для базовых торговых операций и учета карт лояльности с использованием в клиентского (кассового оборудования).
- Offline-First. При наличии стабильной связи запись всегда производится сначала в локальную БД, работающую на самом кассовом аппарате, и только после этого данные отправляются в облако. Промышленным стандартом локальной БД для таких задач является SQLite. При наличии стабильной связи запись о продажах и товарных остатках всегда производится сначала в локальную БД SQLite, и только после этого данные отправляются в БД в облаке процессом синхронизатором. Данный подход позволяет:
 - Избавиться от разработки сложных алгоритмов проверки доступности центральных систем в облаке,
 - Избежать критической зависимости от качества и загрузки каналов связи («фризы» из-за потери пакетов, джиттера, флаппинг и пр).
- Таким образом, локальная SQLite служит буфером и гарантией безопасности. Кассовая программа всегда общается только с SQLite, обеспечивая мгновенный отклик

интерфейса. А синхронизация с облаком реализована в виде изолированного фоновый процесс, который просто «копирует» данные наверх

Лояльность и промо-акции

Для защиты от повторного списания баллов система лояльности должна работать по принципу Online-first, т.е. режим **онлайн** (основной) и **офлайн** (резервный). Выбор режима онлайн режим в качестве основного в случае с системой лояльности допустим, т.к. система является вспомогательной и не применимы регуляторные требования к времени отклика.

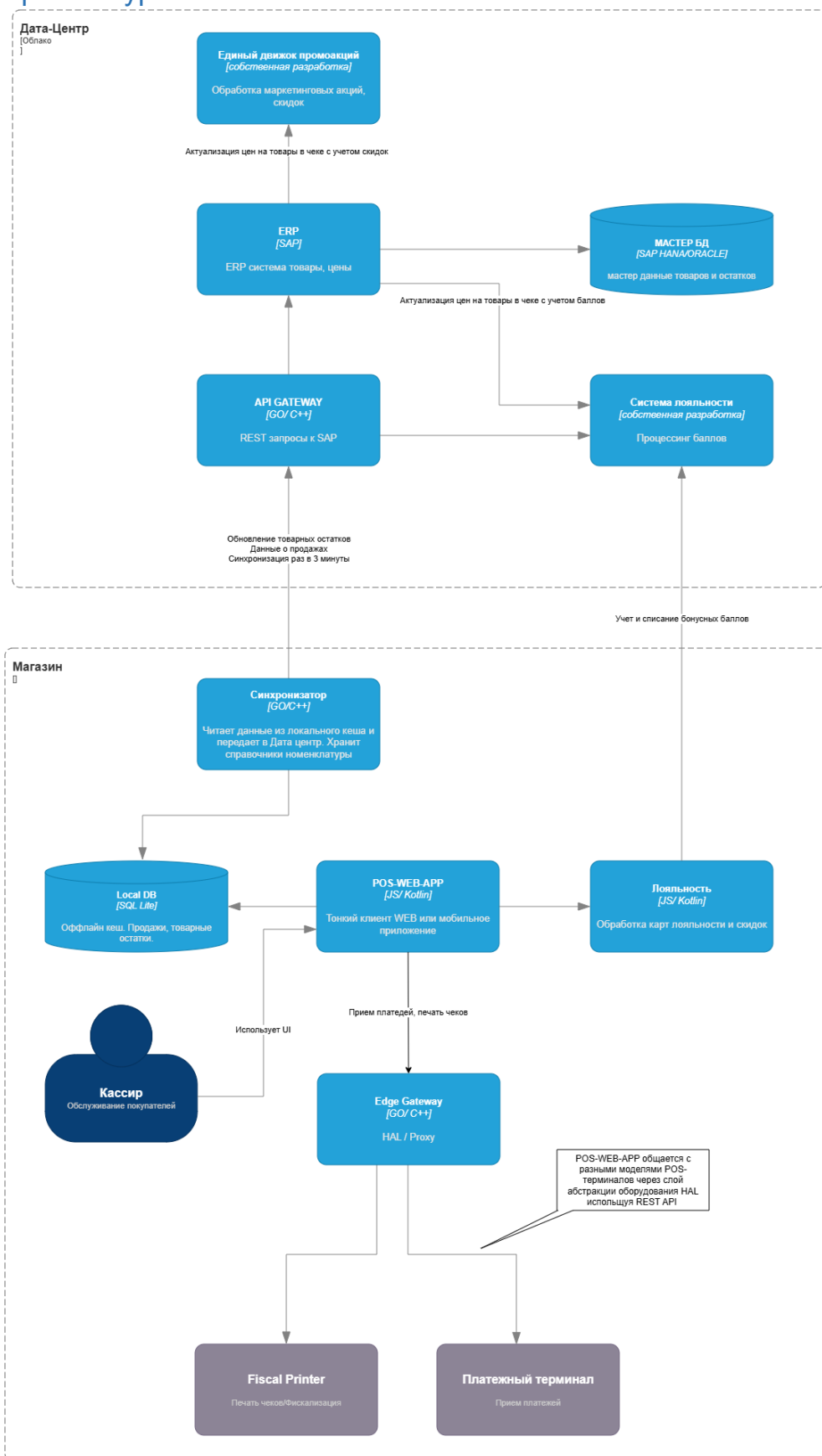
Магазин онлайн (Штатный режим)

- Покупатель предъявляет перед оплатой карту лояльности кассиру или КСО. Касса отправляет запрос на систему лояльности в центральном офисе (облаке).
- Возможно списание или начисление бонусных баллов. Списание баллов происходит мгновенно на сервере в режиме реального времени.

Магазин офлайн (Аварийный режим)

- Касса или КСО работает в офлайн режиме. Покупатель видит сообщение о том, что списание баллов не возможно.
- Покупатель предъявляет перед оплатой карту лояльности кассиру или КСО.
- Касса считает сумму покупки, вычисляет процент к начислению по локальным правилам (например, 5%) и записывает в количество баллов в локальный кеш системы лояльности.
- Возможно только начисление бонусных баллов.
- Бонусные баллы начисляются при восстановлении связи и завершения работы процесса синхронизации локальной БД и данными в облаке.

Целевая архитектура



Общая стратегия миграции от толстого клиента в магазине на облачное решение

Переход выполняется **инкрементально**, чтобы избежать простоев и сохранить возможность работы магазинов в процессе замены компонентов. Стратегия основана на **позапном замещении** модулей, начиная с автоматизации процессов HQ и заканчивая полной заменой кассового ПО.

Используется паттерн Strangler Fig – новый облачный бэкенд постепенно «оборачивает» старые компоненты, пока те полностью не будут заменены.

- Шаг 0 (подготовка): Проведён аудит, выбран стек облачных сервисов, разработаны ML-конвейер маппинга и TCO-модель. Проведена миграция ключевых в облако согласно выбранной стратегии (rehost)
- Шаг 1.1 (пилот): Внедрение Edge Gateway, интеграция с POS терминалами в 5–10 магазинах параллельно с существующей системой.
- Шаг 1.2 (пилот): Внедрение API слоя перед SAP ERP, создание REST-API интерфейсов для веб-касс в 5–10 магазинах параллельно с существующей системой.
- Шаг 1.3 (пилот) внедрение веб-касса с интеграцией с Edge Gateway и SAP ERP в облаке через API слой,
- Шаг 2 (масштабирование): Позапное отключение локальных БД и перевод всех магазинов на синхронизацию с системами в облаке.
- Шаг 3 (завершение): Демонтаж локальных серверов, полный переход на тонкий клиент, отказ от старого POS-приложения.

нтральный сервер лояльности.

1. Кассир сканирует карту лояльности.
2. Касса отправляет запрос через API: GET /loyalty/customer/{id}.
3. Сервер возвращает: ФИО, текущий баланс баллов, уровень скидки и персональные акции.
4. При закрытии чека касса отправляет команду: POST /loyalty/transactions с информацией, сколько баллов **начислить** и сколько **списать**. Списание происходит мгновенно на сервере в режиме реального времени.

Режим 2. Магазин офлайн (Аварийный режим)

Если связь с центральным сервером пропала, касса переходит на автономные правила. Здесь есть два подхода к проектированию:

Вариант А. Консервативный (Безопасный)

- **Начисление баллов:** ✓ Разрешено. Касса считает сумму покупки, вычисляет процент к начислению по локальным правилам (например, 5%) и записывает в чек: *"Начислить 100 баллов"*. При восстановлении сети этот чек уйдет на сервер, и баллы упадут на счет.
- **Списание/Трата баллов:** ✗ Запрещено. Касса физически не знает, сколько баллов у клиента на самом деле. Система выдает сообщение: *"Офлайн-режим. Списание баллов временно недоступно"*. Это защищает бизнес от кражи баллов.

Он считается стандартом для кассовых систем. ⚙ Пошаговый цикл записи (когда интернет есть) Мгновенная локальная фиксация: Кассир нажимает кнопку «Оплатить». Кассовая программа открывает транзакцию и записывает чек в локальную SQLite (это занимает менее 2–5 миллисекунд). Печать чека и выдача сдачи: Поскольку данные уже надежно сохранены на кассовом компьютере, программа сразу же дает команду принтеру печатать чек. Кассир отпускает покупателя. Очереди на кассе не растут. Фоновая отправка в облако: В эту же секунду параллельный фоновый поток (worker) видит, что в SQLite появился новый чек и интернет доступен. Он берет этот чек и отправляет его в облако через API. Подтверждение: Облако принимает чек, сохраняет у себя и возвращает кассе ответ 200 OK. Кассовая программа отмечает в SQLite, что этот чек успешно синхронизирован. ✗ Почему нельзя писать напрямую в облако (даже при хорошем интернете)? Если настроить систему так, чтобы при наличии связи она писала данные напрямую в облако в режиме реального времени, бизнес столкнется со следующими проблемами: «Фризы» и задержки интерфейса: Сетевой запрос в облако и обратно (Round Trip Time) занимает от 50 до 500+ миллисекунд. Кассир будет ощущать микродвижения и задержки при закрытии каждого чека. На локальной SQLite всё происходит моментально. Проблема «микрообрывов» связи: Интернет может пропасть ровно в секунду нажатия кнопки «Оплатить» (например, моргнул роутер или переключилась вышка 4G). Если локальной базы нет, кассовая программа зависнет, выдаст ошибку, а кассир не поймет — ушли деньги или нет, печатать ли чек, и списались ли бонусы. Избыточная нагрузка на сеть: Прямая транзакционная работа с удаленной базой данных по сети генерирует огромный трафик и требует постоянного удержания соединения. 💡 Итог Локальная SQLite служит буфером и гарантией безопасности. Кассовая программа всегда общается только с SQLite, обеспечивая мгновенный отклик интерфейса. А синхронизация с облаком — это изолированный фоновый процесс, который просто «копирует» данные наверх по мере возможности. Чтобы точнее спроектировать этот узел, уточните: Планируется ли в кассовой программе списание бонусов или работа с программами лояльности в реальном времени? (Для этого часто нужен гибридный онлайн-запрос). Нужно ли поддерживать актуальные остатки товаров между кассами в рамках одного магазина в режиме онлайн?

как устроен процесс записи данных Локальная БД (SQLite/Realm/LevelDB): Это обязательный первый рубеж. Касса всегда сначала записывает чек локально. Это гарантирует, что даже если связь оборвется ровно в секунду оплаты, данные не потеряются. Облачный сервер (синхронизация): Сразу после локальной записи (или параллельно ей) кассовое приложение отправляет копию данных в облако по API. Фискальный накопитель (ФН): Параллельно с программной БД, данные о чеке аппаратно записываются в память ФН и уходят к Оператору фискальных данных (ОФД) согласно закону. 📊 Сравнение режимов работы кассы Режим работы кассы Запись в локальную БД (SQLite) Запись/Синхронизация с облаком Причина такого поведения Связь есть (Онлайн) ✓ Да ✓ Да (мгновенно) Локальная база обеспечивает максимальную скорость работы интерфейса, а облако — актуальные отчеты для владельца бизнеса. Связи нет (Офлайн) ✓ Да ✗ Нет (отложена) Касса работает автономно. Все чеки копятся локально и отправятся в облако автоматически, как только сеть восстановится. 💡 Почему кассы не пишут только в

облако?Защита от очередей: Запрос в облако и ответ от него могут занимать от 1 до 5 секунд в зависимости от интернета. Локальная база SQLite отвечает за миллисекунды. Кассиру не приходится ждать загрузки сети при каждом клике.Автономность: Если бы касса писала данные напрямую в облако, любой сбой Wi-Fi или мобильной сети полностью бы парализовал торговлю в магазине.Если вас интересует конкретная кассовая система, уточните название ПО (например, Эвотор, МойСклад, iiko или r_keeper). Я смогу рассказать о технических нюансах её архитектуры подробнее.

Offline-mode strategy:

- **Online (интернет есть):** Web POS работает с облачным API (ЦО), получая актуальные цены и скидки. Edge Gateway периодически кэширует справочник товаров в локальную SQLite. Преимущества БД SQLite: не нужно устанавливать тяжелые СУБД (PostgreSQL, MySQL), SQLite работает прямо внутри кассового приложения.
- **Offline (интернет пропал):** Web POS автоматически переключается в локальный режим – все запросы к ценам и остаткам направляются к Edge Gateway, который использует локальный кэш (БД SQLite). Кассир проводит операцию, Edge Gateway формирует чек с временным UUID и сохраняет его в локальной БД. Фискализация чека происходит мгновенно через кассу, подключенную по COM-или USB порту), что соответствует законодательству.

Механизм синхронизации при восстановлении связи (Reconciliation):

1. Edge Gateway обнаруживает восстановление интернета (Heartbeat-сигнал).
2. Запускается процесс сверки:
 - Edge Gateway отправляет пакет чеков (batch) в Cloud API.
 - Cloud API проверяет дубли по UUID, записывает чеки в Master Database и возвращает подтверждение с реальными ID.
 - Cloud API также передаёт дельту изменений (новые цены, акции), накопившихся в Redis за время простоя.
3. Edge Gateway обновляет локальный кэш, удаляет синхронизированные чеки из очереди и переходит в онлайн-режим.
4. Весь процесс **бесшовен для кассира** – в интерфейсе отображается только индикатор синхронизации, без остановки работы магазина.